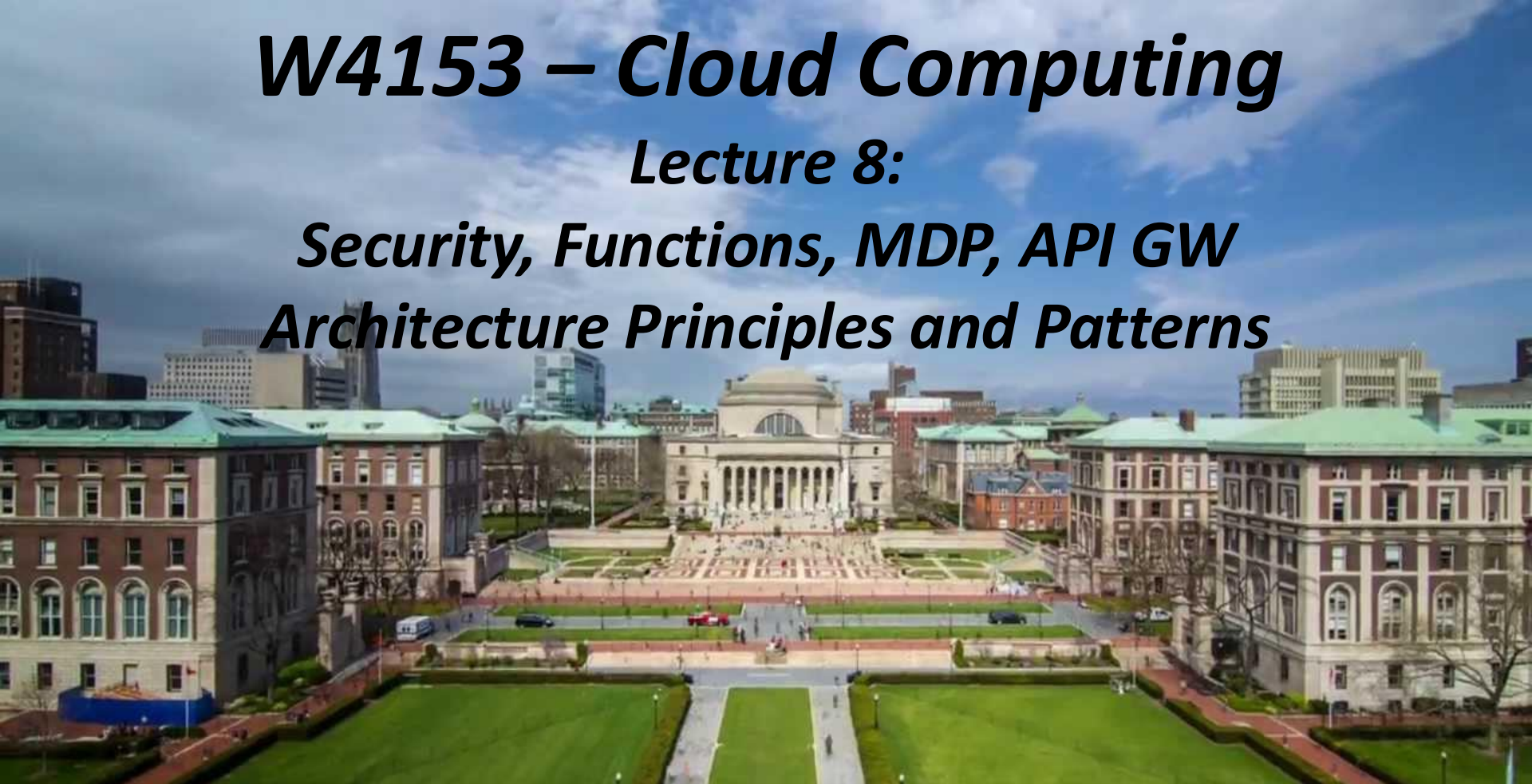


W4153 – Cloud Computing

Lecture 8:

Security, Functions, MDP, API GW Architecture Principles and Patterns



Contents

- Introduction
 - Questions, discussions, course updates
 - Team presentations
- Some technical topics
 - Security: Authorization, Access Tokens and JWT
 - Function-as-a-Service
 - Message Driven Processing
 - Middleware
 - API Gateway
 - Command Query Responsibility Segregation
 - An example of putting some pieces together.
- Architecture patterns and principles, or “Buzz Word Bingo.”

Course Updates

Questions

Discussion

Course Updates, Questions, Discussion

- Sprint 2 Deadline is Sunday, November 23 at 11:59 PM ET.
See Ed for details (<https://edstem.org/us/courses/86008/discussion/7309064>)
- There will be one more sprint, but no status report.
Sprint 3 reporting is the final project review.
- Final project review
 - There will be a checklist of concepts you learned in class that you must apply.
 - I will publish the checklist and discuss in the last lecture.
 - Submission format:
 - A final written report documenting what you did, the implementation architecture,
 - A presentation documenting what you did, the implementation architecture,
 - You will present to the TA focal point/mentor for your team via Zoom, Hangouts,
This will be a 20-minute e-meeting.
 - You will arrange the presentation time with a TA.
 - Presentations will occur between 07-Dec and 13-Dec.
- I still owe you a lecture. I am on vacation this coming week, which means I will have time to complete. I will do as early in the week as possible.

Questions? Discussion?

Team Presentations

Team Presentations

Team	Focal Point
Blue Cloud	Sai Dinesh Chintaginjala
Untitled	Sahasra Kokkula

Security Continued

- *Authorization*
- *JWT and tokens*

Concepts

Authentication vs Authorization

Authentication:

- Verifies who a user or system is.
- Goal: Confirm identity before granting access.
- Examples: Passwords, biometrics, OAuth, certificates.
- Occurs first in the security process.

Authorization:

- Determines what an authenticated user can do.
- Goal: Control access to resources or actions.
- Examples: Role-based access control (RBAC), ACLs, policies.
- Happens after authentication and depends on it.

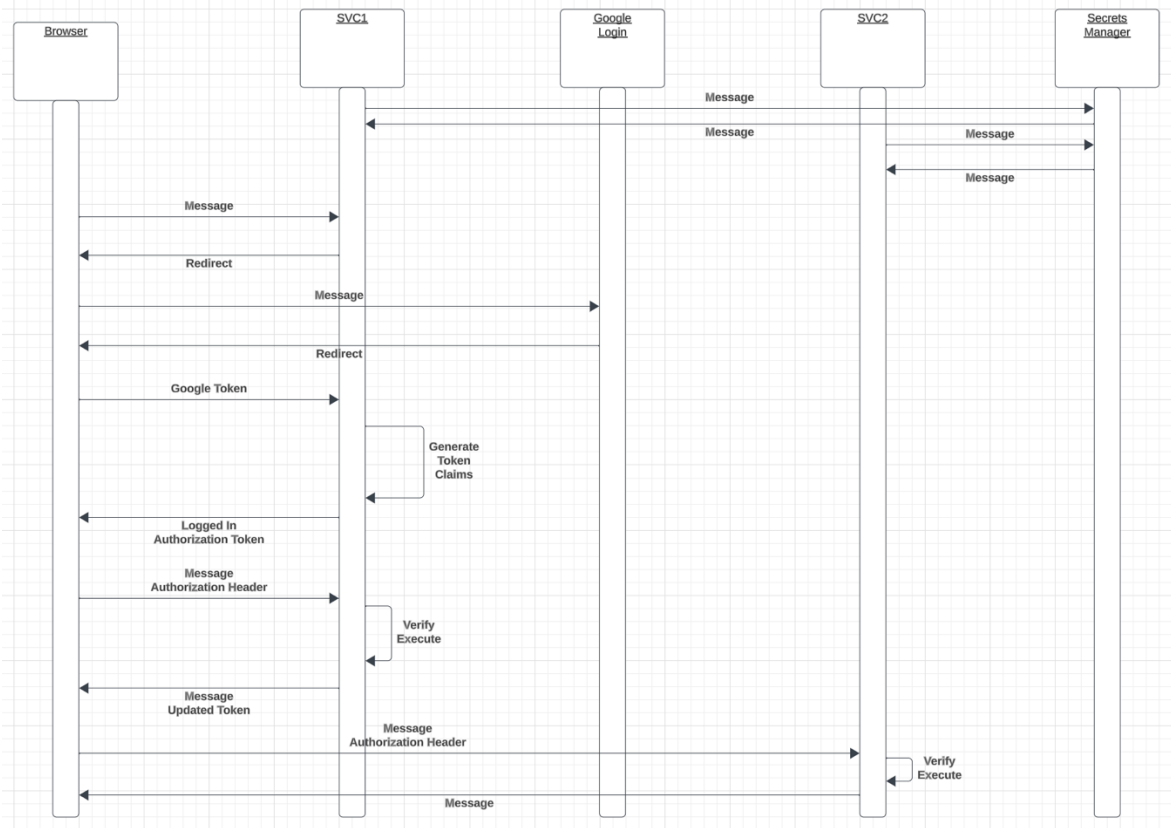
Concepts

- Even when relying on OIDC/OAuth2, cloud/microservices almost always implement or use:
 - A user profile service that defines user properties, groups, roles,
 - An authorization service that defines which
 - Which roles
 - Can perform which operations
 - Based on which resources
 - Overly simplistically, a table of the form:
 {resource path, [role1, role2, ...], [operation1, operation2, ...]}
 - The operations are usually CREATE, RETRIEVE, UPDATE, DELETE
- After successful login, the “security service” creates a domain specific access token that contains user properties, including roles.
- On each operation on a resource, middleware calls the “security service” to determine if the user is authorized to perform the operation.
- There are commercial off the shelf-security service products.

Some Terms

- An access token claim is a piece of information asserted about a user or application that is included in an access token. These claims are used by a resource server (like an API) to authorize access, with examples including the user's ID (sub claim), the token's intended audience (aud claim), and the permissions granted (scope claim). Access tokens are for authorization, not authentication, and should not be used to determine if a user is logged in.
- The token issuing service *signs the claim* using a public key. The signature allows a recipient to validate that the token service issued the token/claim and that it is not “forged.”
- Think of an access token claim being like a permission slip your teacher gave you in high-school to be “in the hall outside of class.”
 - Signature verifies the issuer.
 - The scope is what the permission slip allows the client to do.

Login and Authorization Tokens

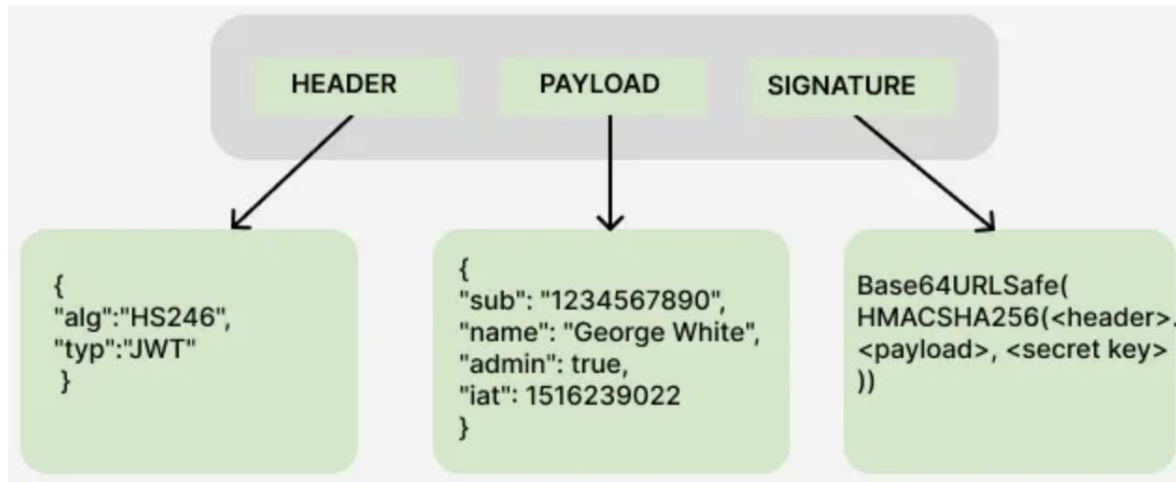


- We have seen Google Login.
- Forms the basics of authentication.
- The server applications:
 - "Look up" user info in their database and systems.
 - Determine what operations the user can perform.
 - Generate a JWT Authorization containing signed claims.
 - Returns the token in an authorization header.
- The client
 - "Saves" the token.
 - Attached to subsequent calls.
- Servers can then authorize based on
 - Verifying the signature.
 - Checking that the client has a claim authorizing the operation.

JASON Web Tokens (JWT)

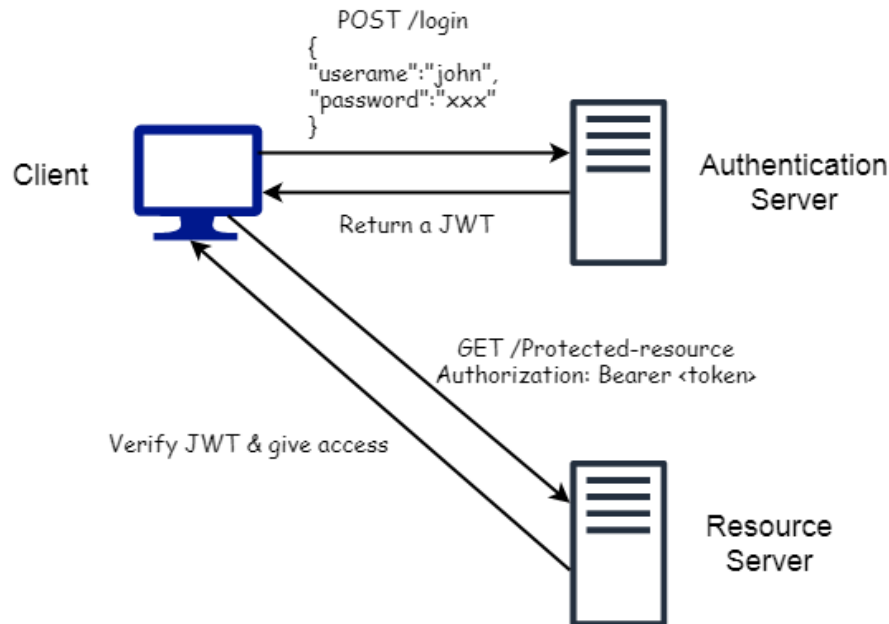
JWT

- A JSON Web Token (JWT) is an open standard for securely transmitting information between parties as a digitally signed JSON object. It consists of three parts—a header, a payload, and a signature—separated by dots, which are Base64URL encoded. JWTs are used to verify the integrity of claims (information about a user) and are commonly used for [authentication](#) and authorization in web applications, mobile apps, and APIs.



JWT

1. A user logs in to an application, which then creates a JWT.
The application basically has become a *secure token service(er)*.
2. The JWT is
 1. Signed with a private key or a secret key to prove its authenticity.
 2. Contains one or more claims about the user.
3. The application sends the JWT back to the client, which can then include it in subsequent requests to access protected resources.
4. The server receives the token
 1. Uses the public key or secret key to verify the signature.
 2. If the signature is valid, the server trusts the information in the payload and processes the request without needing to look up user information again.
 3. The “authorization service” validates that the claims *authorize* the user to perform the operation on the resource.



When should you use JSON Web Tokens?

Here are some scenarios where JSON Web Tokens are useful:

- **Authorization:** This is the most common scenario for using JWT. Once the user is logged in, each subsequent request will include the JWT, allowing the user to access routes, services, and resources that are permitted with that token. Single Sign On is a feature that widely uses JWT nowadays, because of its small overhead and its ability to be easily used across different domains.
- **Information Exchange:** JSON Web Tokens are a good way of securely transmitting information between parties. Because JWTs can be signed—for example, using public/private key pairs—you can be sure the senders are who they say they are. Additionally, as the signature is calculated using the header and the payload, you can also verify that the content hasn't been tampered with.

<https://medium.com/swlh/all-you-need-to-know-about-json-web-token-jwt-8a5d6131157f>

Algorithm

HS256



Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwidGVhbV9yaWdodHMiOnsidGVhbSI6Iksnb2wiLCJyb2x1IjoiaWoiQWRtaW4ifSwiaWF0IjoxeNTE2MjM5MDIyfQ.cfrW-UXR4e-nPqoIgp0IhmD05IknP0202EE
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

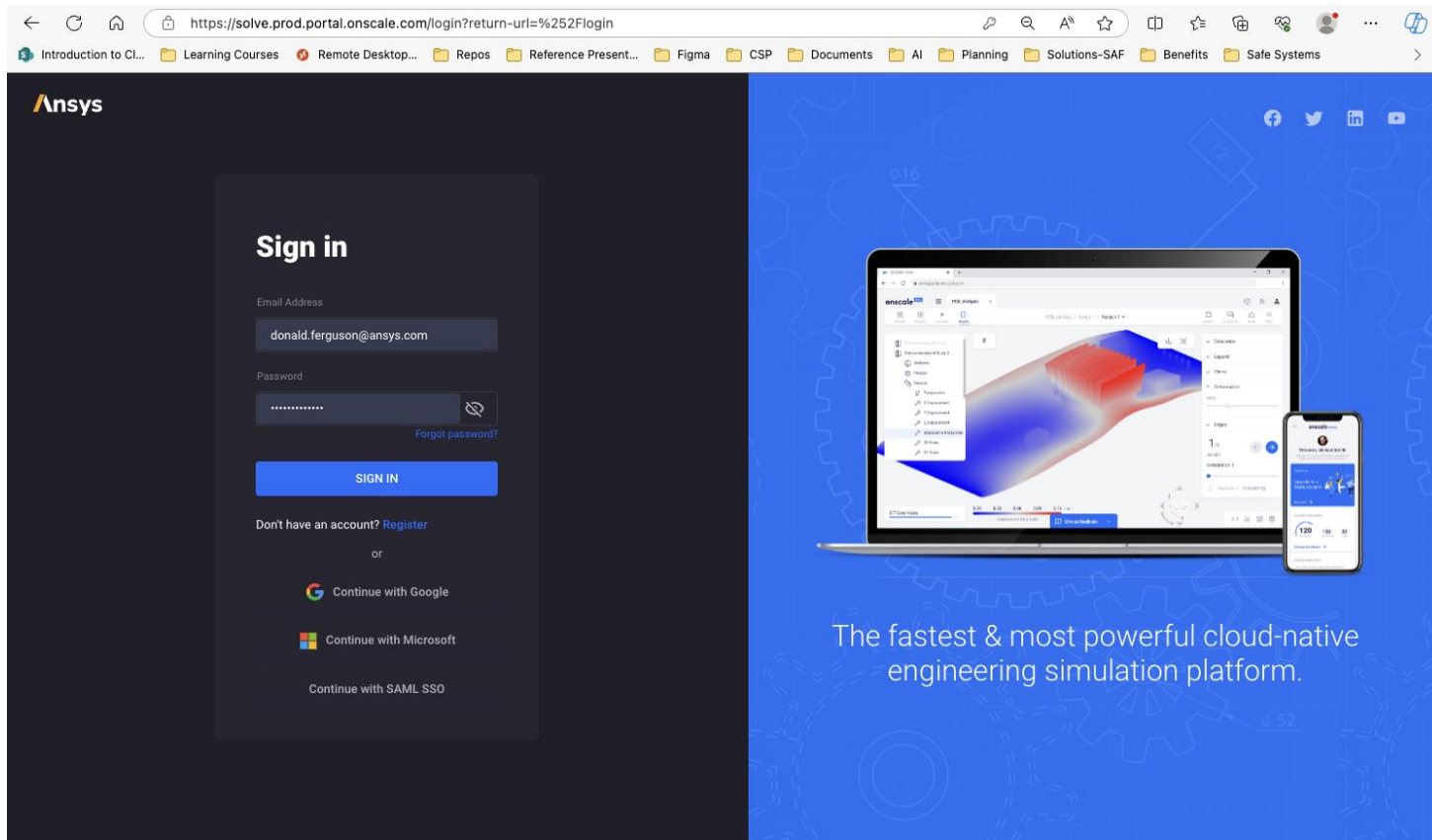
PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "team_rights": {"team": "Cool", "role": "Admin"},
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

Real Example



Real Example

[illegible]

JWT

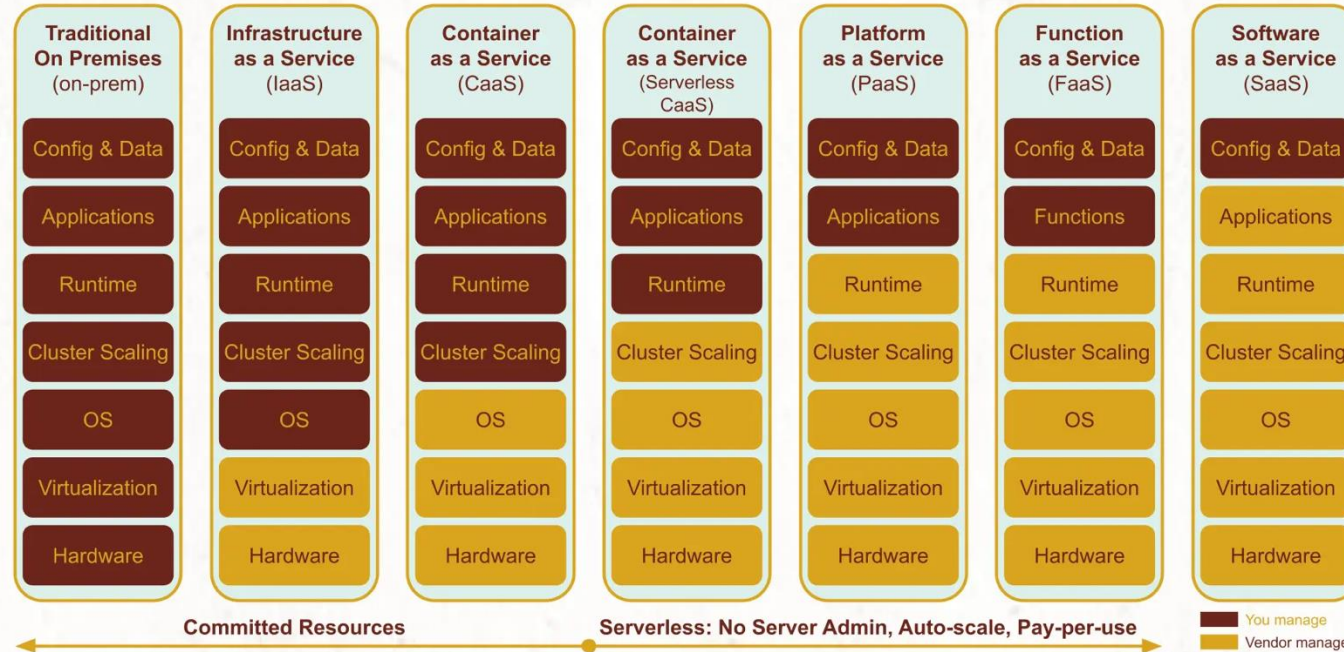
- Sample tool <https://www.jwt.io/>

Function-as-a-Service

Cloud Layers

Cloud Deployment Spectrum

ml4devs.com/serverless 



 © Satish Chandra Gupta, Creative Commons BY-NC-ND 4.0 International License.

scgupta 

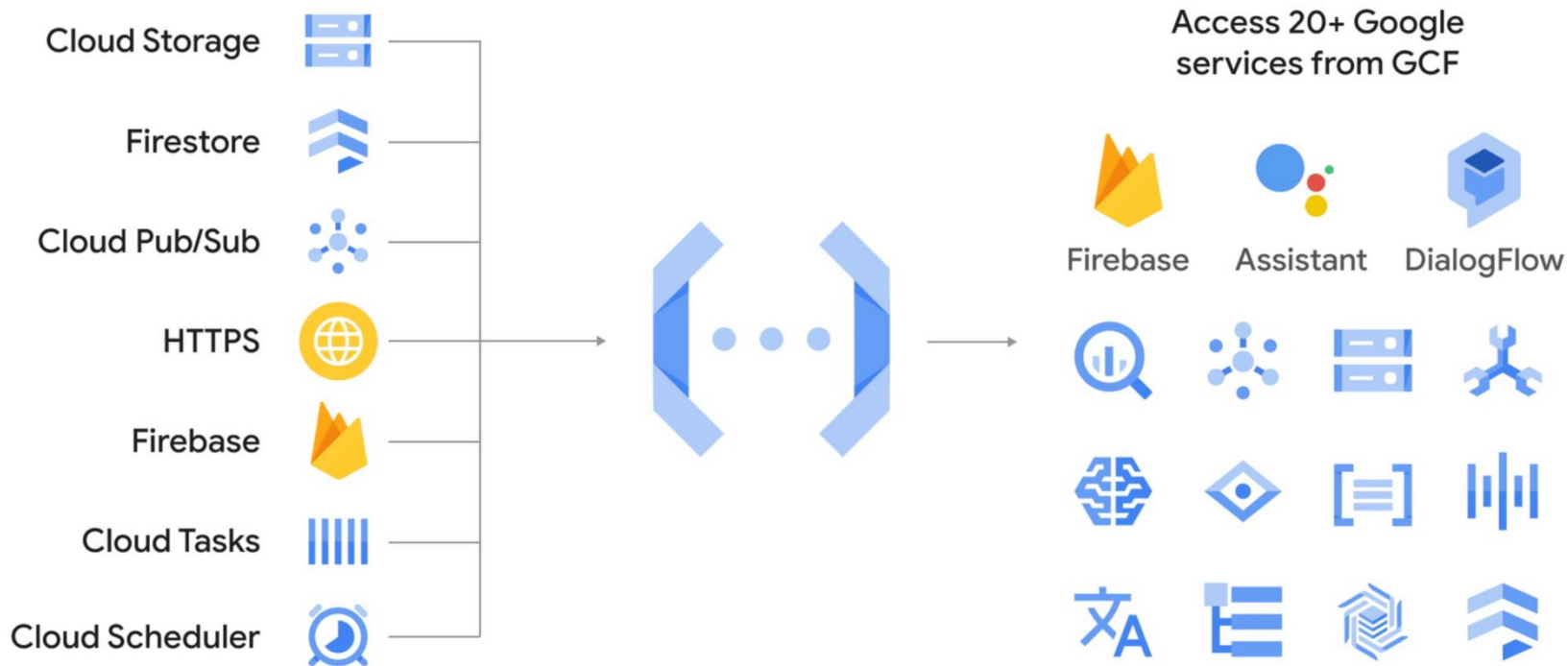
linkedin.com/in/scgupta 

Function-as-a-Service Principles

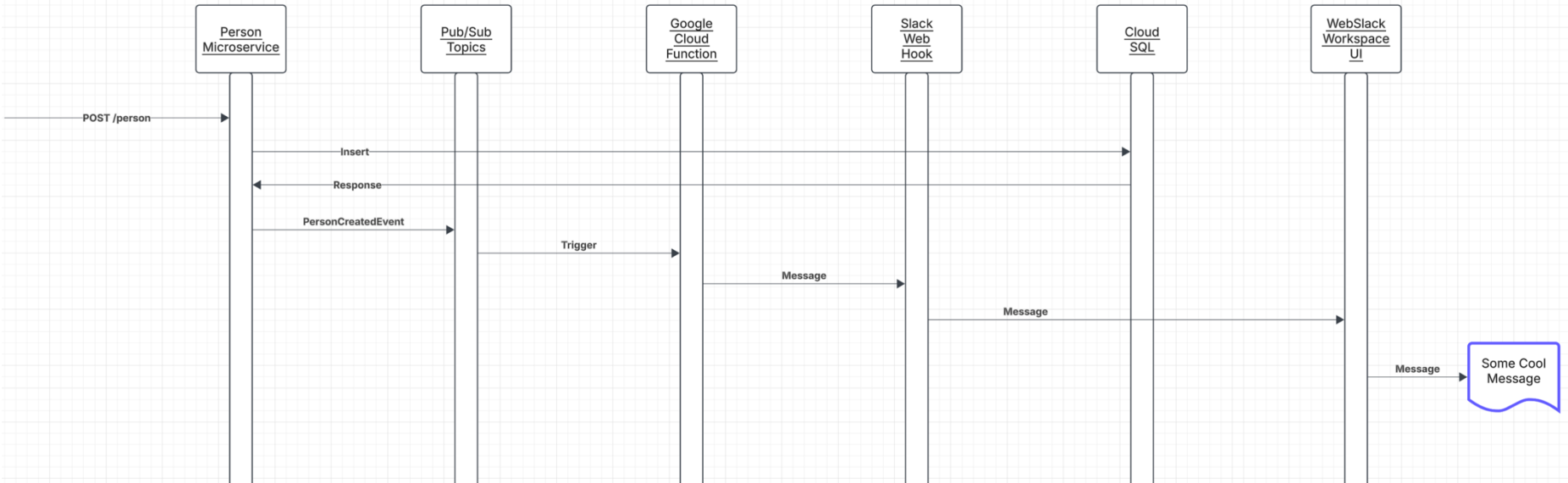
- Event-Driven Execution:
 - Specific events trigger FaaS functions. These events can be an HTTP request, a message in a queue, database update, scheduled task,
 - The corresponding function executes when an event occurs.
 - Think of this being like the generalization of a database trigger.
- Stateless Functions: Each function is self-contained. It maintains no state between invocations, enabling easy scaling and parallel execution.
- Short-Lived Execution: Functions run for a short duration, typically seconds to minutes. The system creates them when needed and removes them once their task is complete, optimizing resource use.
- Managed Infrastructure: The cloud provider handles the underlying infrastructure, including servers, scaling, and security. It abstracts the functional complexities away from the developers.

Google Cloud Functions

Cloud Functions: the glue that connects cloud services



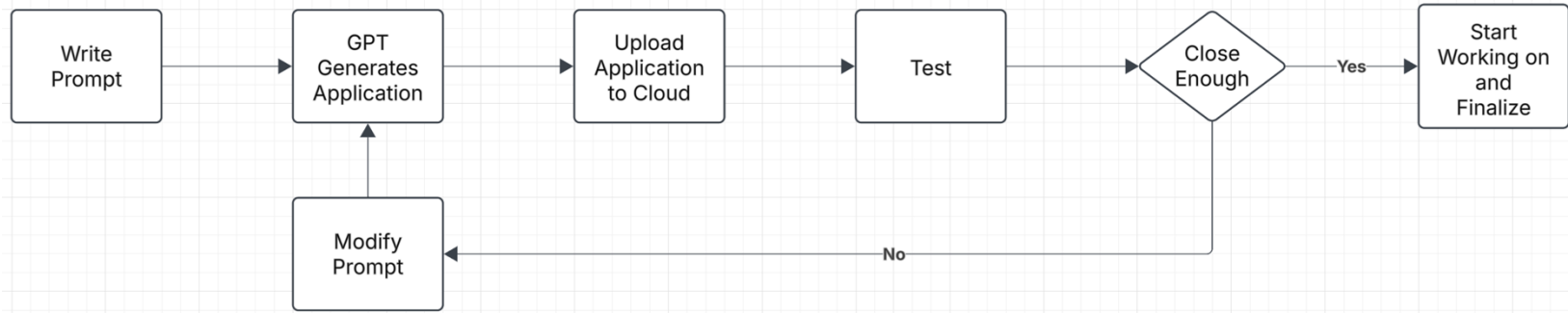
Scenario



- For now, just focus on plumbing everything together.
- You do not need to connect the microservice. You can just trigger with gcloud CLI.
- Collaboration/Social Media space:
 - I will use either Slack or Discord. You can choose whatever you want.
 - For now, you can just write to the Google log to check and demonstrate that your function ran.

Project Walkthrough

- `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/simple_html`
- Not tested on GCP yet. Will do this weekend, but ...
 - This is a simple ChatGPT generated template/starter project.
 - The new style of programming is ...



- You can/should do the same.

Show Code

- `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repo/slack-pubsub-notify`
- `/Users/donald.ferguson/Dropbox/000/000-A-2025-Repo/pubsub-hello 2`
- Where are we going with this ...
 - We are going to learn about pub/sub and events.
 - We will expand our composition example to use pub/sub and functions.
 - We will also include some form of “user notification” in our project using functions and pub/sub
 - And seeing the complexity, we will look at workflow and state machines

Message Driven Processing

Message Queues

Message Driven Processing

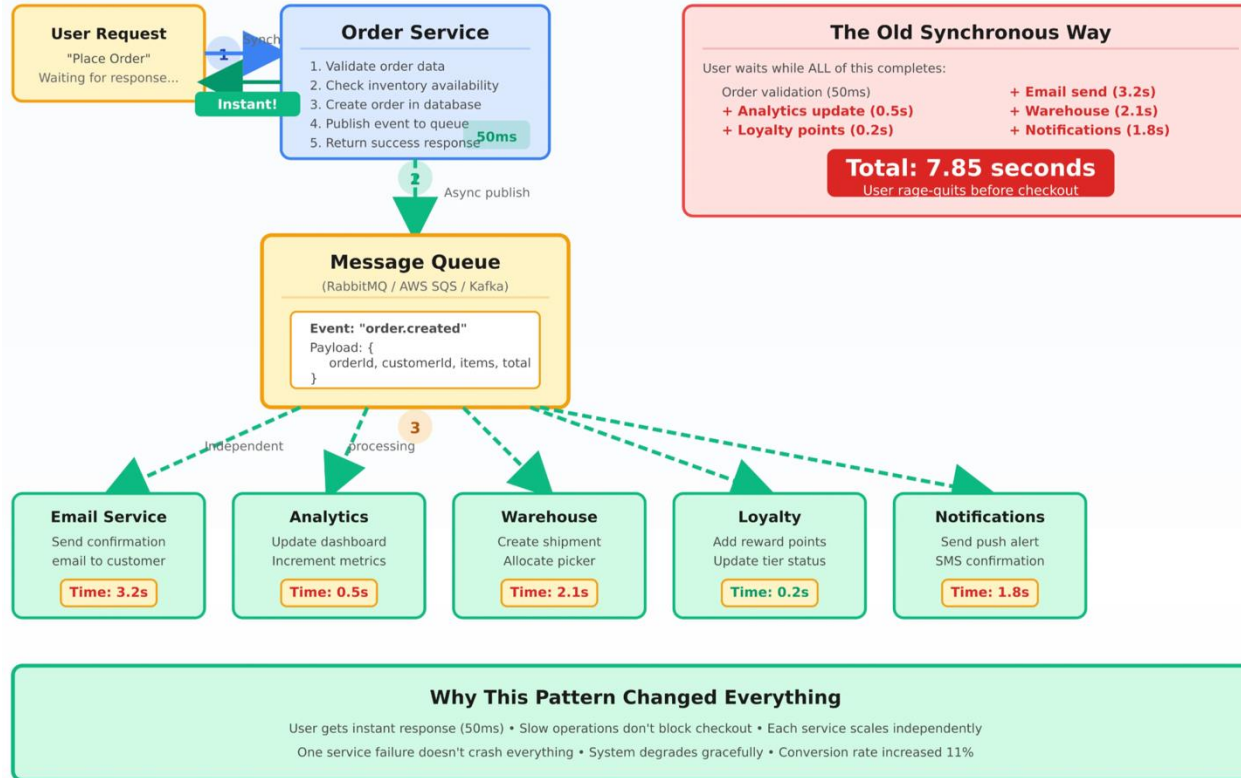
- Message-driven processing is an asynchronous communication pattern where components exchange messages through a messaging system, such as a message broker. Instead of direct calls, one component (a producer) sends a message and another component (a consumer) processes it later, allowing for loose coupling and decoupling of services. This approach enhances reliability and scalability by allowing systems to process work independently and at their own pace.
- Key concepts
 - Asynchronous communication: The sender does not wait for an immediate response from the receiver. The sender places a message and continues with its work.
 - Producer and consumer: The producer creates and sends the message, while the consumer receives and processes the message from a queue or topic.
 - Decoupling: Components do not need to know about each other's internal workings, only about the message format and the messaging system. This allows for independent development and scaling.
 - Messaging system: An intermediary, like a message broker, stores messages and ensures they are delivered from producers to consumers. Examples include using a command bus for commands and an event bus for events.
 - Reliability and resilience: If a consumer is temporarily unavailable, the message can be held in the messaging system until the consumer is ready to process it. This makes the system more resilient to failures
- Very similar to event-driven-processing.

Simple Examples

- **Example: Order processing** – Imagine an online store's order system.
 - When a customer places an order, the system sends an "OrderPlaced" message to a message queue.
 - Other services, like the inventory service, shipping service, and billing service, can subscribe to this queue.
 - Each service picks up the message and performs its task independently:
 - The inventory service decrements the stock.
 - The billing service processes the payment.
 - The shipping service prepares the package.
 - This is more reliable and scalable than if one service had to directly call all the others, especially if one of them is slow or fails
- **Message-driven vs. event-driven**
 - While often used together, there's a subtle difference:
 - Message-driven: Focuses on commands or work to be done. The message is a "to-do" item for a specific consumer or a group of consumers.
 - Event-driven: Focuses on events that have occurred. A message containing an event is broadcast to multiple consumers who are interested in that event, such as a "UserSignedUp" event.
 - Many modern systems use a hybrid approach, combining message-driven for transactional operations with event-driven for broader system notifications

Event-Driven Architecture: From 4 Seconds to 50ms

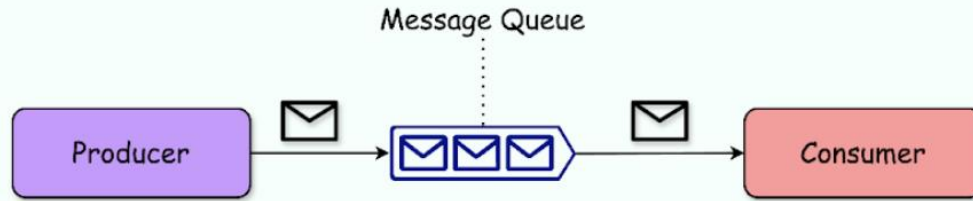
How Black Friday orders went from crawling to instant



Message Queuing

A Cheatsheet On **Message Queues**

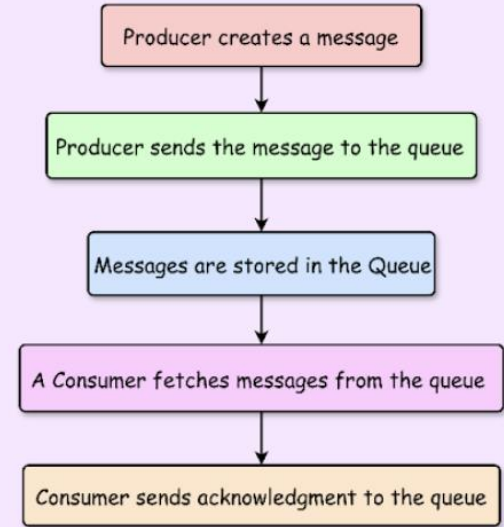
1 What is a Message Queue?



BENEFITS

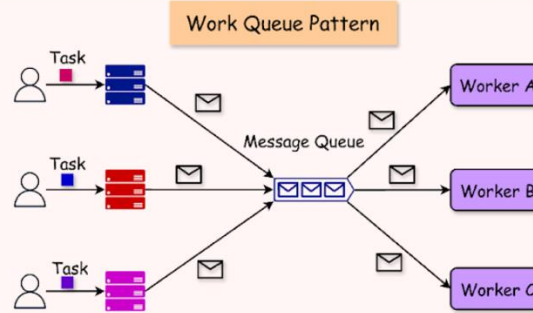
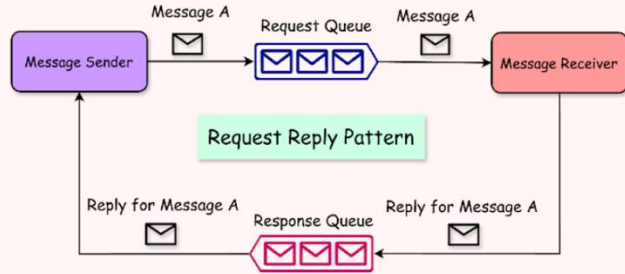
- ✓ Decoupling
- ✓ Scalability
- ✓ Fault Tolerance
- ✓ Async Processing
- ✓ Resiliency

2 How Message Queue Operates?

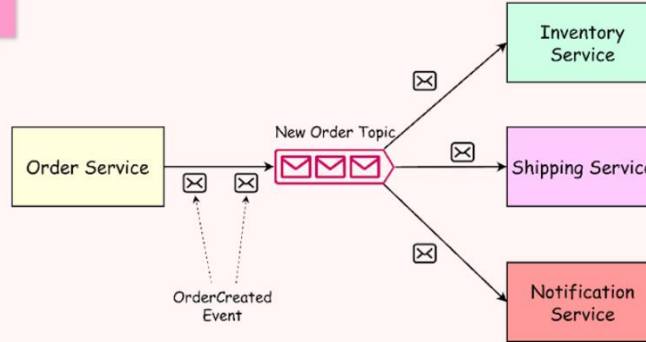


Message Queuing

3 Message Queue Patterns



Publish Subscribe Pattern



Message Queueing and Pub/Sub

- Conceptually, a message queue is
 - Ordered (first in first out)
 - Reliable
 - Often a transactional resource, i.e. it is possible to “commit.”
 - The implication is that some resource is listening or will eventually listen on the queue and process the messages in order.
- Pub/Sub
 - Is simply shouting, “XXX happened” on a topic.
 - Zero, one or many resources may be subscribed to the topic.
 - Events are not necessarily ordered or reliable.
- Most systems, including Google Cloud Platform, mingles the two sets of concepts into a single implementation paradigm. GCP optionally adds FIFO, guaranteed, dead letter, to Google Pub/Sub

Middleware

Middleware

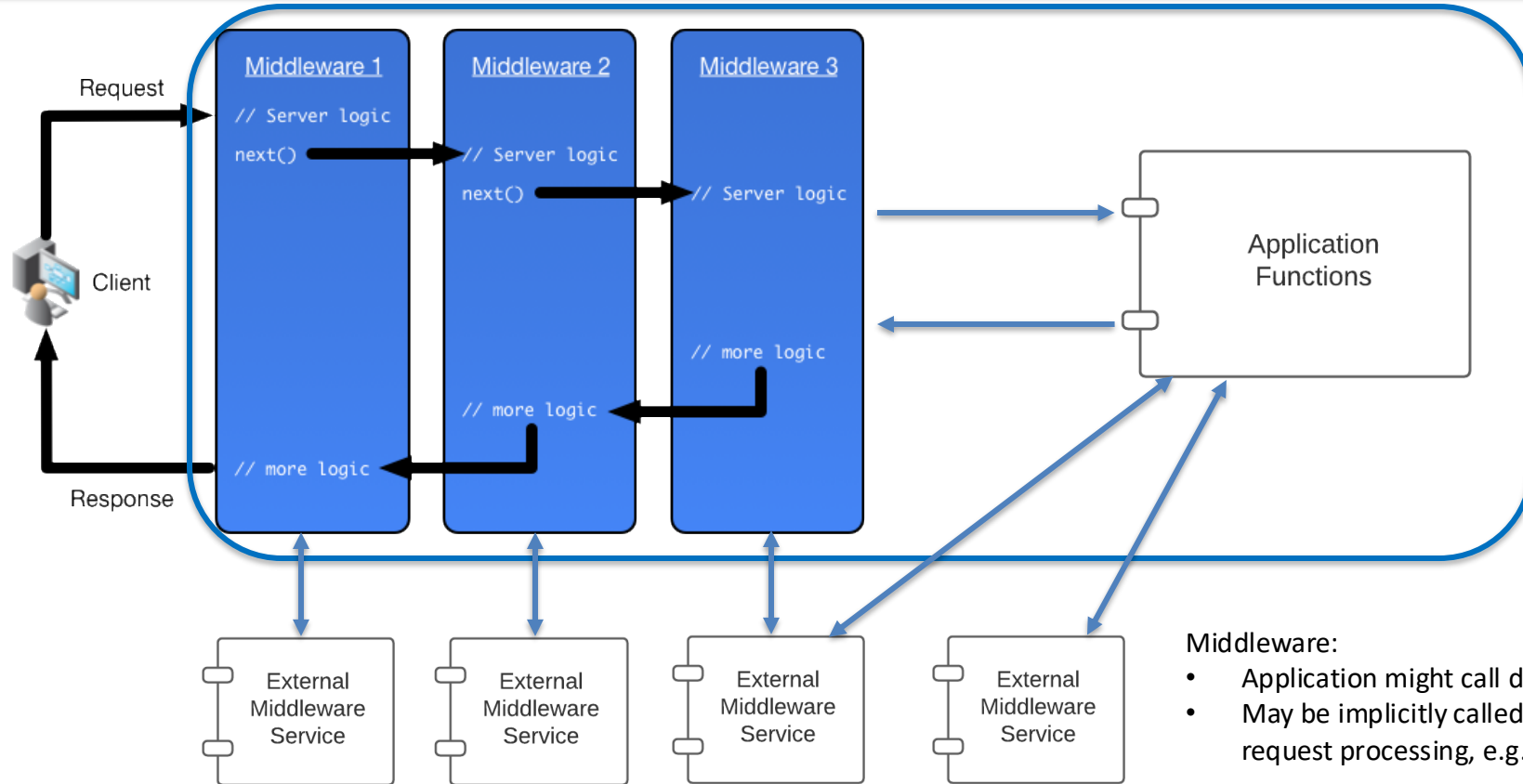
[Overview](#) / What is middleware?

What is middleware?

Middleware is software that lies between an operating system and the applications running on it. Essentially functioning as hidden translation layer, middleware enables communication and data management for distributed applications. It's sometimes called plumbing, as it connects two applications together so data and databases can be easily passed between the "pipe." Using middleware allows users to perform such requests as submitting forms on a web browser, or allowing the web server to return dynamic web pages based on a user's profile.

Common middleware examples include database middleware, application server middleware, message-oriented middleware, web middleware, and transaction-processing monitors. Each program typically provides messaging services so that different applications can communicate using messaging frameworks like simple object access protocol (SOAP), web services, representational state transfer (REST), and JavaScript object notation (JSON). While all middleware performs communication functions, the type a company chooses to use will depend on what service is being used and what type of information needs to be communicated. This can include security authentication, transaction management, message queues, applications servers, web servers, and directories. Middleware can also be used for distributed processing with actions occurring in real time rather than sending data back and forth.

Middleware – Conceptual Model



Middleware:

- Application might call directly, e.g. DB
- May be implicitly called as part of request processing, e.g. logging.

Middleware – Conceptual Model



- This is another example of
 - Single Responsibility
 - Separation of Concerns
- Being a good application logic developer and a good middleware developer are different roles, skill sets,
- Also an example of the
 - Closed for modification, but
 - Open for extension



- Application might call directly, e.g. DB
- May be implicitly called as part of request processing, e.g. logging.

What Does Middleware Do?

1. **Incoming Request:** When a request comes in, the middleware can examine and even modify parts of the request like cookies, headers, query parameters, etc., before it reaches the actual API service method. For example, it can validate the request, authenticate the user, or log the request details.
2. **Outgoing Response:** After the API service method has processed the request and generated a response, the middleware gets a chance to inspect and modify the response before it goes back to the client. This can include things like changing the response headers, transforming the content, and so on.

FastAPI Middleware

Middleware

<https://fastapi.tiangolo.com/tutorial/middleware/>
<https://fastapi.tiangolo.com/advanced/middleware/>

You can add middleware to **FastAPI** applications.

A "middleware" is a function that works with every **request** before it is processed by any specific *path operation*. And also with every **response** before returning it.

- It takes each **request** that comes to your application.
- It can then do something to that **request** or run any needed code.
- Then it passes the **request** to be processed by the rest of the application (by some *path operation*).
- It then takes the **response** generated by the application (by some *path operation*).
- It can do something to that **response** or run any needed code.
- Then it returns the **response**.

Example

- Walk through Medium Article
- <https://medium.com/@saverio3107/mastering-middleware-in-fastapi-from-basic-implementation-to-route-based-strategies-d62eff6b5463>
- This is the very simple version.
There is a more sophisticated approach to middleware in FastAPI.
<https://fastapi.tiangolo.com/advanced/middleware/>

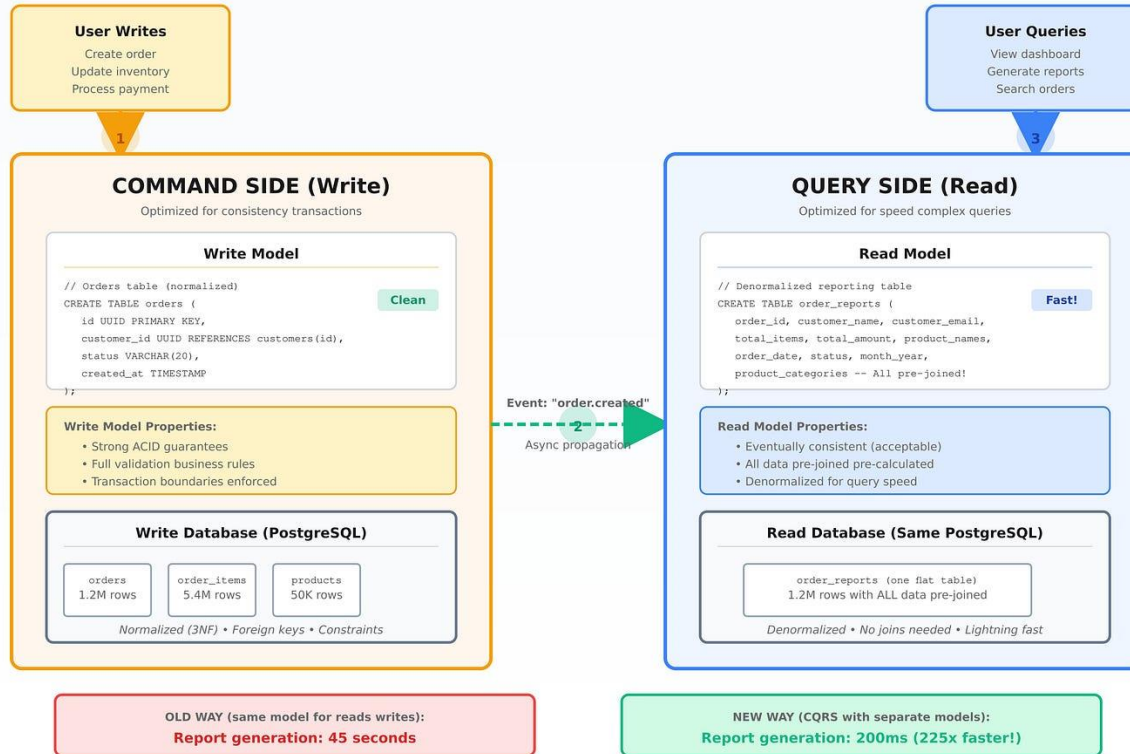
CQRS

- CQRS, or Command Query Responsibility Segregation, is a design pattern that separates a system's write operations (commands) from its read operations (queries). This separation allows the two models to be independently designed, optimized, and scaled, improving performance and handling complex domains more effectively. The pattern uses different models for commands and queries and can be combined with [event sourcing](#), where commands are turned into events that then update optimized read models.
 - Separate Models: The write side uses a model optimized for writing, while the read side uses a model optimized for reading, often in the form of a denormalized view or database.
 - Independent Scaling: Because the read and write sides are distinct, they can be scaled independently. For example, a high-volume read-heavy system can scale its read models without affecting the write operations.
 - Flexibility and Optimization: Each model can use a different data store that is best suited for its purpose, leading to better performance and easier optimization.
 - Complex Domains: CQRS is particularly useful in complex domains where the logic for writing data is different from the logic for reading it.

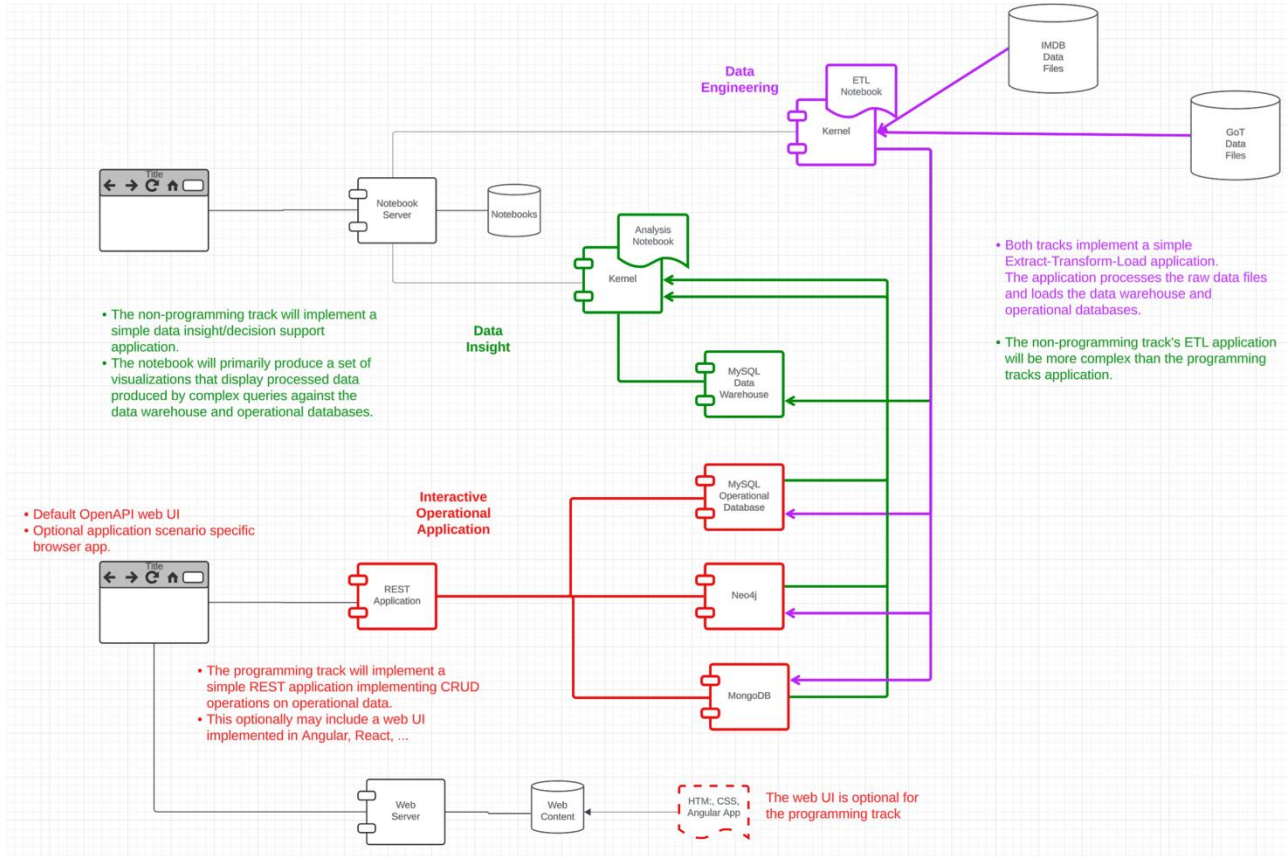
- Advantages of using CQRS
 - Improved scalability and performance: Separating reads and writes allows for fine-tuning each side for its specific needs.
 - Clearer code structure: The separation creates clear boundaries and makes it easier to focus on specific business logic without getting bogged down in the whole system's complexity.
 - Reduced complexity: By keeping related code together in silos, it can simplify modifying or creating new code.
 - Greater flexibility: It is easier to make architectural changes or optimizations to one part of the system without impacting the other.
- Potential drawbacks of using CQRS
 - Increased complexity: Implementing CQRS can add complexity to a project, especially for simpler applications.
 - Eventual consistency: Keeping the read and write models in sync can be challenging and may lead to eventual consistency, where the read model is not immediately up-to-date after a write.

CQRS: From 45 Seconds to 200ms

How separate read/write models transformed our reporting system



Hypothetical Example

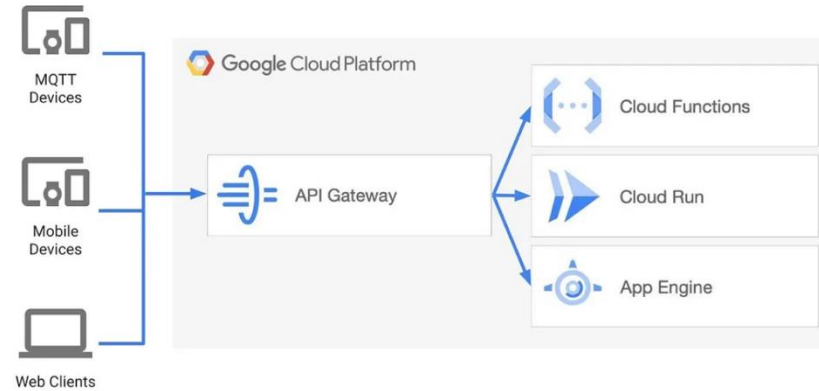
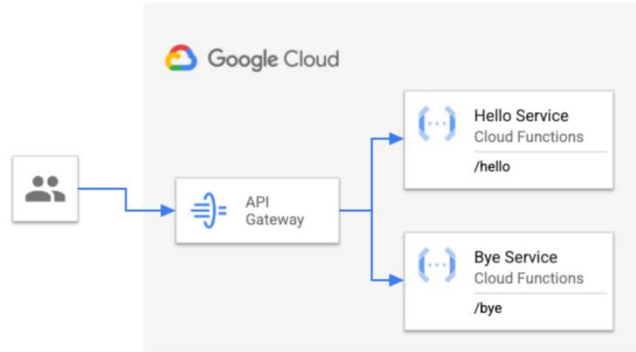


API Gateway

API Gateway

- An API gateway is a server that acts as a single entry point for clients interacting with backend microservices, handling requests, and routing them to the appropriate services. It simplifies application development and management by centralizing functions like security, authentication, rate limiting, and monitoring. By managing traffic, the gateway decouples the client interface from the complex backend, improving scalability and reliability.
- Key functions
 - Request routing: Directs client requests to the correct backend service.
 - Protocol translation: Can translate between different communication protocols.
 - Security enforcement: Handles authentication, authorization, and other security policies.
 - Load balancing: Distributes traffic across multiple instances of a service.
 - Monitoring and logging: Aggregates logs and metrics for analysis.
 - Caching: Stores and returns responses to reduce the load on backend services.
 - Request/response transformation: Modifies requests or responses as they pass through the gateway.

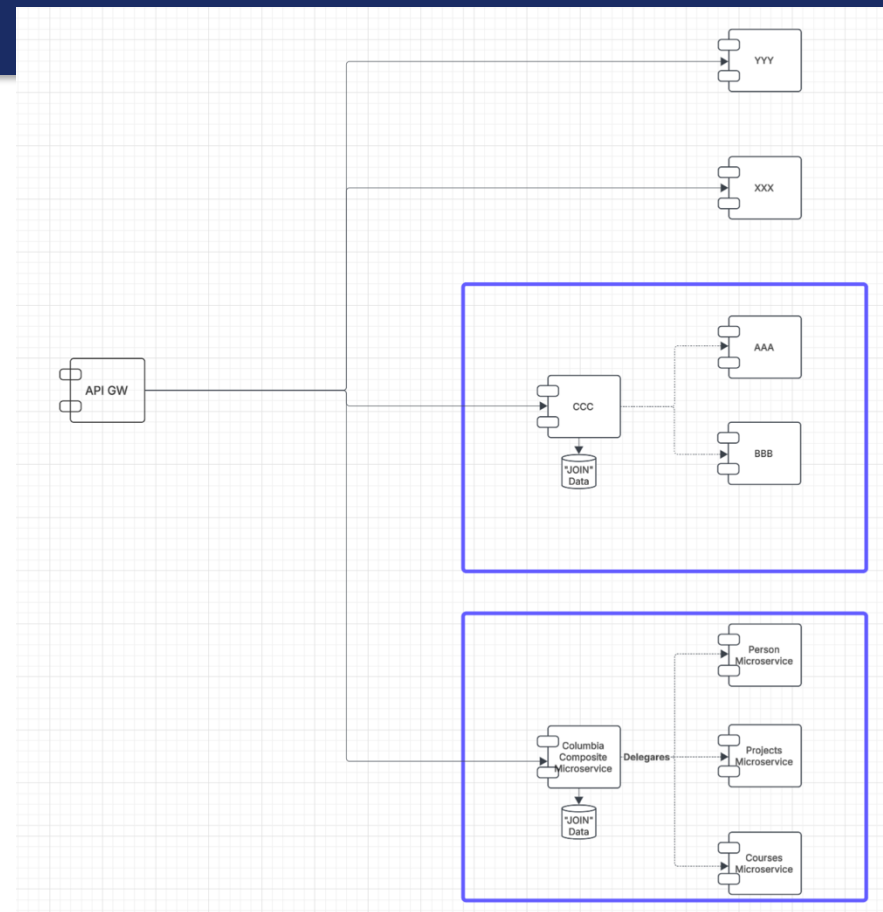
API Gateway – Some Benefits



- **Simplifies client development:** Clients only need to interact with a single API gateway, not a multitude of microservices.
- **Decouples frontends and backends:** Allows backend services to be refactored, scaled, or replaced without affecting the client.
- **Centralizes cross-cutting concerns:** Manages common tasks like security and monitoring in one place, rather than repeating them in every microservice.
- **Improves security:** Enforces security policies at the edge of the network, protecting internal services.

API GW and composite

- API Gateway and Composite microservices implement different functions and concerns.
- The composite
 - Assembles independently developed microservices into a consistent application.
 - Implements domain functions.
- API Gateway
 - Provides a single endpoint over all of an enterprise's applications.
 - Injects cross-cutting concerns and QoS functions, e.g. authorization, rate limiting,



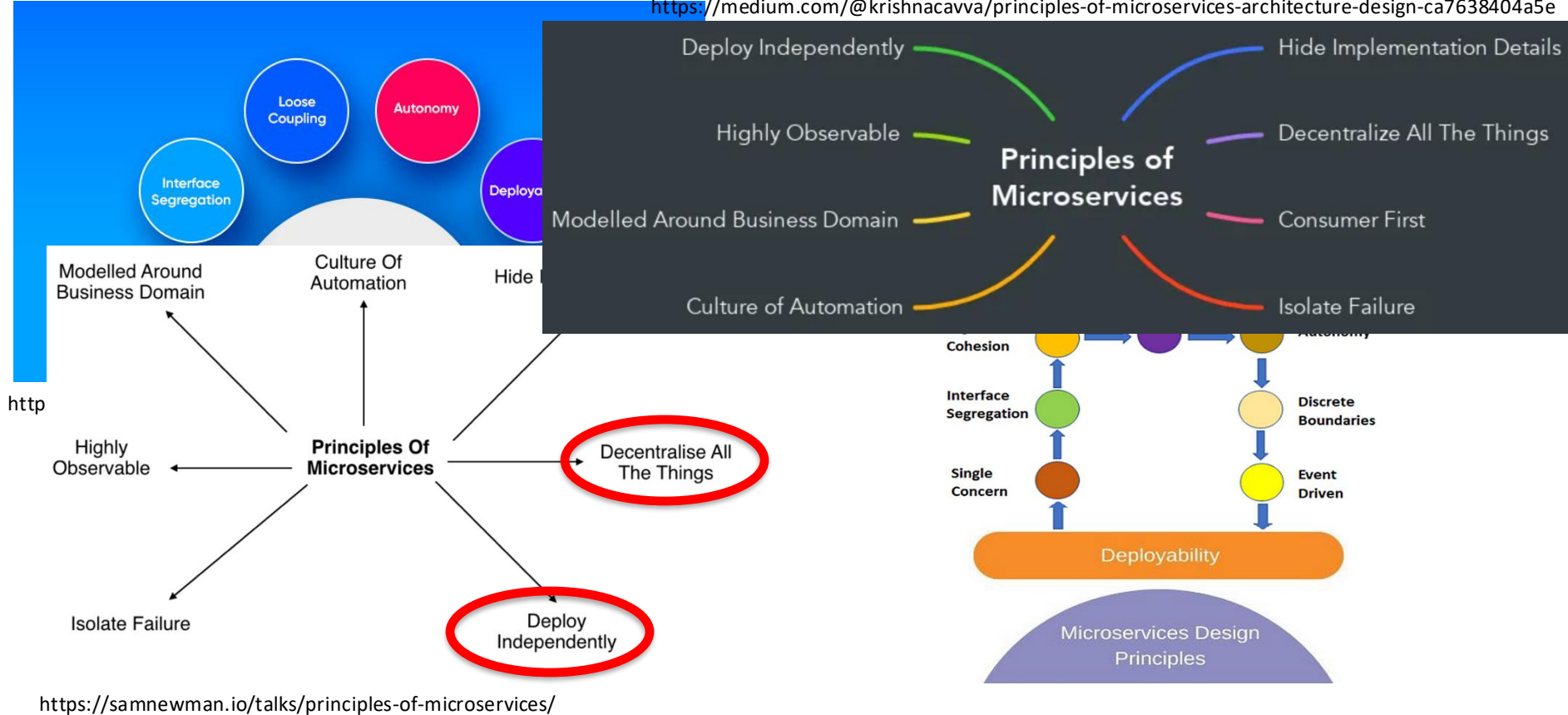
An Example Putting Some Pieces Together

Architecture Principles and Design Patterns

Modular Monolith and Microservices

Microservice Principles

<https://medium.com/@krishnacavva/principles-of-microservices-architecture-design-ca7638404a5e>



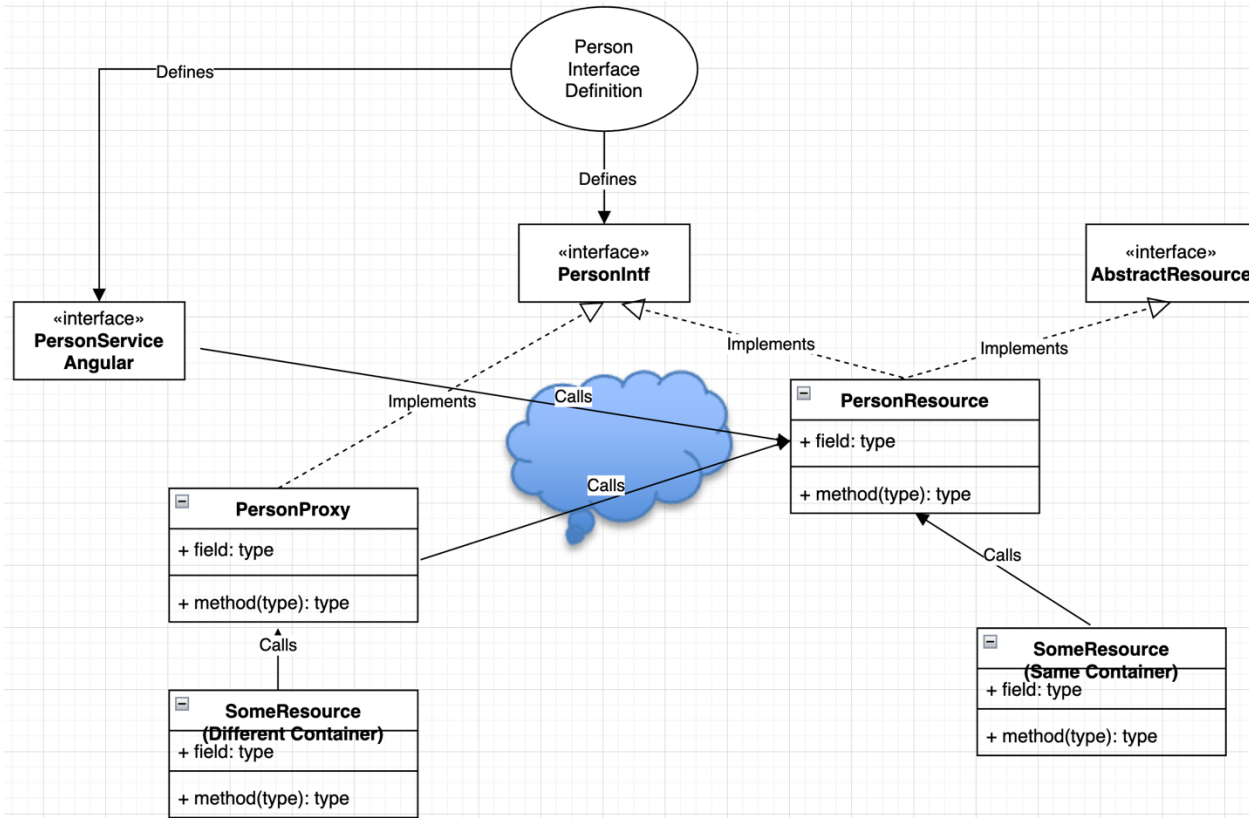
Microservice Principles

- **Deploy Independently:** “One of the fundamental principles of Microservices architecture is the ability to deploy each service independently. This enables teams to release updates and bug fixes without impacting other services, fostering agility and reducing time-to-market. By decoupling deployment pipelines, teams can iterate rapidly and respond to changing business requirements more effectively.”
- **Decentralize Everything:** “Decentralization is a core principle in Microservices architecture, empowering teams to make autonomous decisions and innovate at their own pace. Rather than relying on centralized governance and coordination, decentralized architectures distribute decision-making authority, fostering a culture of ownership and experimentation. Microservices teams are responsible for the full lifecycle of their services, from design and development to deployment and operations, enabling faster innovation and adaptation.”

My Philosophy is

- (Try to Enable)
 - Deploy independently
 - Decentralize everything
- Do not preclude deploying “together” with a *remote-local* programming model.
- The core concepts
 - Think in terms of *modules following architecture principles*.
 - Models
 - Well-defined resource interface (abstract base class)
 - Resource implementation
 - Dependency injection
 - Modules go into *logical containers*. There may be more than one *module in a container*. The container is comprised from:
 - A (web) application sever framework augmented by “my framework.” A well-defined directory/package structure for “packaging” a module/modules in the web application.
 - A “Docker” container.
 - A resource calls another resource directly if in the same module, otherwise uses a *proxy*.
 - Rely on my customization of *design patterns*, e.g. dependency injection, service factory, service locator,

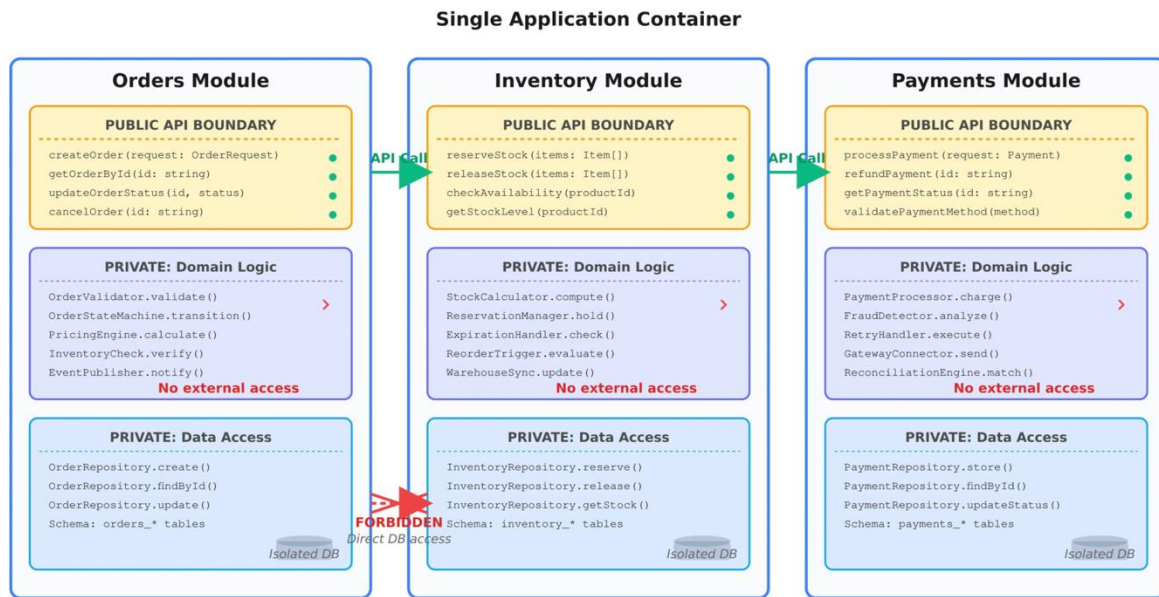
My Philosophy is



This Approach Enables a Modular Monolith

The Modular Monolith: Instagram's Architecture

Single deployment, clear boundaries, 2 billion users



This approach enables “future proofing.”

Module may be separated into separate “microservices” containers later.

Separate “microservices” can, under certain constraints, be remapped to the same container.

The Golden Rules of Modular Monoliths

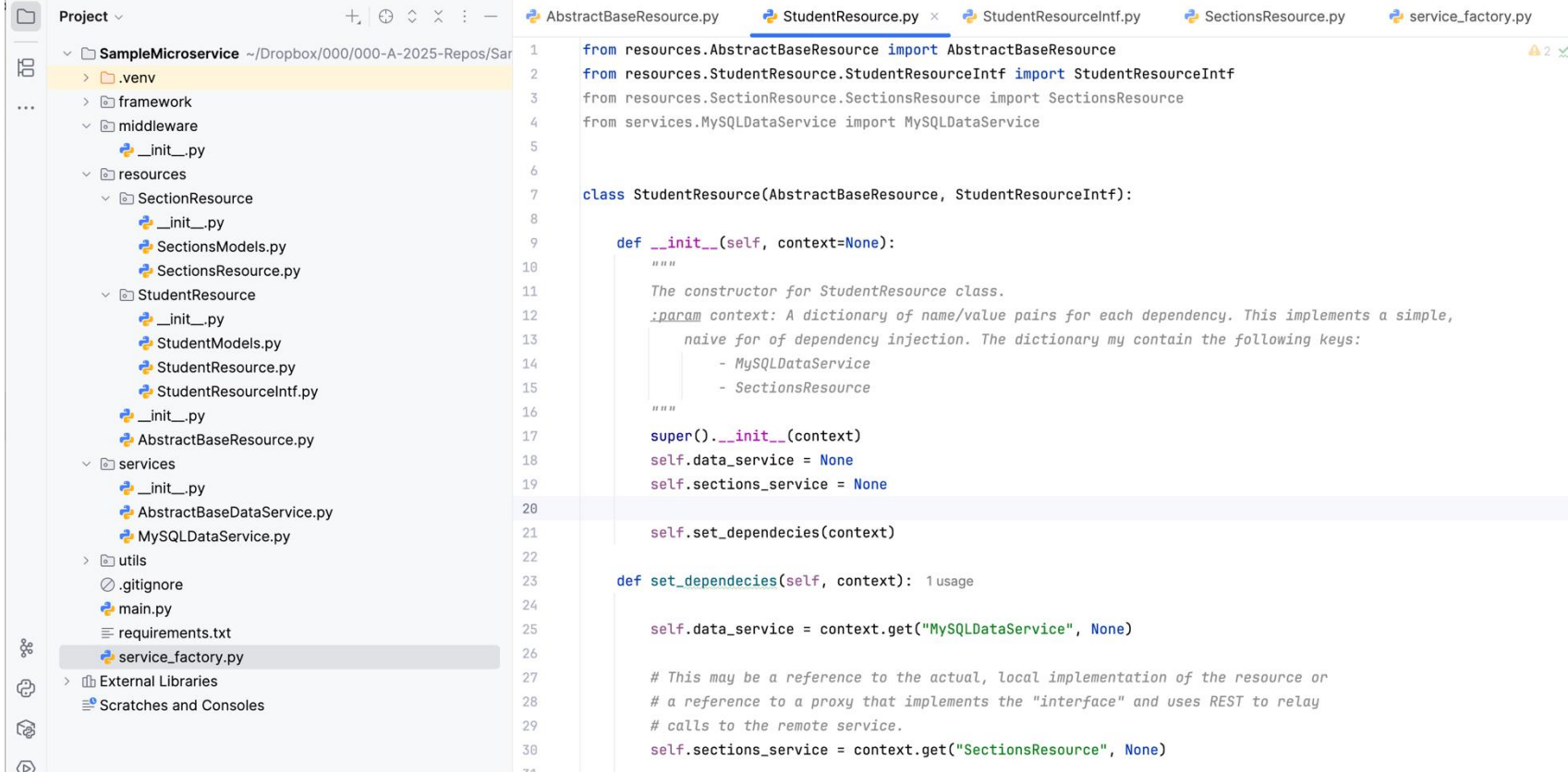
Modules communicate **ONLY** through public APIs • Each module owns its database schema • No internal class imports • Enforce with linting rules • One deployment, clear boundaries

Dependency Injection Service Factory

Dependency Injection/Service Factory

- *Dependency injection* (DI) is a design pattern where an object receives other objects (its dependencies) from an external source rather than creating them itself. This approach decouples the components of an application, making code more reusable, testable, and maintainable. Dependencies are typically provided through constructor, setter, or interface injection.
- *A Service Factory*
 - Is the first thing that *main.py* calls.
 - The implementation
 - Creates the concrete instances of all resources. The implementation may be the actual resource implementation or a proxy depending on the configuration choices.
 - Creates the concrete instances of all services.
 - Injects dependencies into implementations by passing in a dictionary of key/value pairs:
 - Service or resource name
 - Implementation class

Dependency Injection/Service Factory



The screenshot displays a code editor interface. On the left, a 'Project' sidebar shows a directory tree for 'SampleMicroservice'. The tree includes folders for '.venv', 'framework', 'middleware', 'resources', and 'services'. The 'resources' folder is expanded, showing 'SectionResource' and 'StudentResource'. The 'StudentResource' folder is further expanded, showing files like 'StudentModels.py', 'StudentResource.py', 'StudentResourceIntf.py', and 'AbstractBaseResource.py'. The 'services' folder is also expanded, showing 'AbstractBaseDataService.py' and 'MySQLDataService.py'. The 'utils' folder contains '.gitignore', 'main.py', and 'requirements.txt'. The 'service_factory.py' file is highlighted in the 'utils' folder.

The main editor area shows the code for 'StudentResourceIntf.py'. The code defines a class 'StudentResource' that inherits from 'AbstractBaseResource' and 'StudentResourceIntf'. The class has an '.__init__' method that takes a 'context' parameter and sets 'self.data_service' and 'self.sections_service' to 'None'. It also has a 'set_dependencies' method that takes a 'context' parameter and sets 'self.data_service' and 'self.sections_service' to the values returned by 'context.get' for 'MySQLDataService' and 'SectionsResource' respectively.

```
1 from resources.AbstractBaseResource import AbstractBaseResource
2 from resources.StudentResource.StudentResourceIntf import StudentResourceIntf
3 from resources.SectionResource.SectionsResource import SectionsResource
4 from services.MySQLDataService import MySQLDataService
5
6
7 class StudentResource(AbstractBaseResource, StudentResourceIntf):
8
9     def __init__(self, context=None):
10
11         """
12         The constructor for StudentResource class.
13         :param context: A dictionary of name/value pairs for each dependency. This implements a simple,
14             naive for of dependency injection. The dictionary my contain the following keys:
15             - MySQLDataService
16             - SectionsResource
17         """
18         super().__init__(context)
19         self.data_service = None
20         self.sections_service = None
21
22     def set_dependencies(self, context):
23
24         self.data_service = context.get("MySQLDataService", None)
25
26         # This may be a reference to the actual, local implementation of the resource or
27         # a reference to a proxy that implements the "interface" and uses REST to relay
28         # calls to the remote service.
29         self.sections_service = context.get("SectionsResource", None)
```

/Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/SampleMicroservice/resources/StudentResource/StudentResource.py

Summary 1

Summary and Implications

- Weinberg's Second Law: "If builders built buildings the way programmers wrote programs, then the first woodpecker that came along would destroy civilization."
- Summary
 - There is a difference between software engineering and hacking by free associating and writing code like a slam poet on a triple espresso.
 - You do not need to use these patterns but should understand that
 - Real microservices and projects have a complex structure.
 - This implies
 - A well-thought-out directory and package structure.
 - Some framework for componentization and flexible assembly.
 - Most teams and teams of teams define some reference architecture for building and assembling modules, microservices and solutions. Most reference architectures have similar abstractions but with differing implementation details and choices.

12 Factor Applications

12 Factor Applications

<https://dzone.com/articles/12-factor-app-principles-and-cloud-native-microser>



12 Factor App Principles

Codebase

One codebase tracked in revision control, many deploys

Dependencies

Explicitly declare and isolate the dependencies

Config

Store configurations in an environment

Backing Services

Treat backing resources as attached resources

Build, release, and, Run

Strictly separate build and run stages

Processes

Execute the app as one or more stateless processes

Port Binding

Export services via port binding

Concurrency

Scale-out via the process model

Disposability

Maximize the robustness with fast startup and graceful shutdown

Dev/prod parity

Keep development, staging, and production as similar as possible

Logs

Treat logs as event streams

Admin processes

Run admin/management tasks as one-off processes

Dependencies

II. Dependencies

Explicitly declare and isolate dependencies

Most programming languages offer a packaging system for distributing support libraries, such as CPAN for Perl or Rubygems for Ruby. Libraries installed through a packaging system can be installed system-wide (known as “site packages”) or scoped into the directory containing the app (known as “vendoring” or “bundling”).

A **twelve-factor app never relies on implicit existence of system-wide packages**. It declares all dependencies, completely and exactly, via a *dependency declaration* manifest. Furthermore, it uses a *dependency isolation* tool during execution to ensure that no implicit dependencies “leak in” from the surrounding system. The full and explicit dependency specification is applied uniformly to both production and development.

For example, Bundler for Ruby offers the `Gemfile` manifest format for dependency declaration and `bundle exec` for dependency isolation. In Python there are two separate tools for these steps – Pip is used for declaration and Virtualenv for isolation. Even C has Autoconf for dependency declaration, and static linking can provide dependency isolation. No matter what the toolchain, dependency declaration and isolation must always be used together – only one or the other is not sufficient to satisfy twelve-factor.

One benefit of explicit dependency declaration is that it simplifies setup for developers new to the app. The new developer can check out the app’s codebase onto their development machine, requiring only the language runtime and dependency manager installed as prerequisites. They will be able to set up everything needed to run the app’s code with a deterministic *build command*. For example, the build command for Ruby/Bundler is `bundle install`, while for Clojure/Leiningen it is `lein deps`.

Twelve-factor apps also do not rely on the implicit existence of any system tools. Examples include shelling out to ImageMagick or `curl`. While these tools may exist on many or even most systems, there is no guarantee that they will exist on all systems where the app may run in the future, or whether the version found on a future system will be compatible with the app. If the app needs to shell out to a system tool, that tool should be vendored into the app.

- Most application frameworks have a dependency declaration concept.
- I have been showing examples with:
 - Python import
 - requirements.txt
 - etc.
- My examples also show encapsulating libraries and APIs with Python classes.
- II. Dependencies handles “the code,” but we still need to handle the instance. There is a difference between, e.g.
 - pymysql
 - A specific connection.

III. Configurations

III. Config

Store config in the environment

An app's *config* is everything that is likely to vary between deploys (staging, production, developer environments, etc). This includes:

- Resource handles to the database, Memcached, and other backing services
- Credentials to external services such as Amazon S3 or Twitter
- Per-deploy values such as the canonical hostname for the deploy

Apps sometimes store config as constants in the code. This is a violation of twelve-factor, which requires **strict separation of config from code**. Config varies substantially across deploys, code does not.

A litmus test for whether an app has all config correctly factored out of the code is whether the codebase could be made open source at any moment, without compromising any credentials.

Note that this definition of “config” does **not** include internal application config, such as `config/routes.rb` in Rails, or how code modules are connected in Spring. This type of config does not vary between deploys, and so is best done in the code.

Another approach to config is the use of config files which are not checked into revision control, such as `config/database.yml` in Rails. This is a huge improvement over using constants which are checked into the code repo, but still has weaknesses: it's easy to mistakenly check in a config file to the repo; there is a tendency for config files to be scattered about in different places and different formats, making it hard to see and manage all the config in one place. Further, these formats tend to be language- or framework-specific.

The twelve-factor app stores config in *environment variables* (often shortened to *env vars* or *env*). Env vars are easy to change between deploys without changing any code; unlike config files, there is little chance of them being checked into the code repo accidentally; and unlike custom config files, or other config mechanisms such as Java System Properties, they are a language- and OS-agnostic standard.

Another aspect of config management is grouping. Sometimes apps batch config into named groups (often called “environments”) named after specific deploys, such as the `development`, `test`, and `production` environments in Rails. This method does not scale cleanly: as more deploys of the app are created, new environment names are necessary, such as `staging` or `qa`. As the project grows further, developers may add their own special environments like `joes-staging`, resulting in a combinatorial explosion of config which makes managing deploys of the app very brittle.

In a twelve-factor app, env vars are granular controls, each fully orthogonal to other env vars. They are never grouped together as “environments”, but instead are independently managed for each deploy. This is a model that scales up smoothly as the app naturally expands into more deploys over its lifetime.

- Each of the deployment environments we will use has a method for configuring environment variables.
- CI/CD, GitHub actions, etc. can integrate environment variables with the development process.
- Vaults and secrets managers are a better approach for passwords.

IV. Backing Services

IV. Backing services

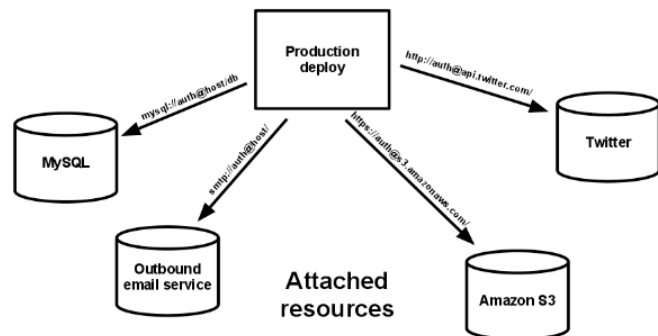
Treat backing services as attached resources

A *backing service* is any service the app consumes over the network as part of its normal operation. Examples include datastores (such as MySQL or CouchDB), messaging/queueing systems (such as RabbitMQ or Beanstalkd), SMTP services for outbound email (such as Postfix), and caching systems (such as Memcached).

Backing services like the database are traditionally managed by the same systems administrators who deploy the app's runtime. In addition to these locally-managed services, the app may also have services provided and managed by third parties. Examples include SMTP services (such as Postmark), metrics-gathering services (such as New Relic or Loggly), binary asset services (such as Amazon S3), and even API-accessible consumer services (such as Twitter, Google Maps, or Last.fm).

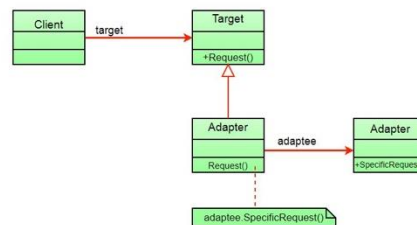
The code for a twelve-factor app makes no distinction between local and third party services. To the app, both are attached resources, accessed via a URL or other locator/credentials stored in the `config`. A deploy of the twelve-factor app should be able to swap out a local MySQL database with one managed by a third party (such as Amazon RDS) without any changes to the app's code. Likewise, a local SMTP server could be swapped with a third-party SMTP service (such as Postmark) without code changes. In both cases, only the resource handle in the config needs to change.

Each distinct backing service is a *resource*. For example, a MySQL database is a resource; two MySQL databases (used for sharding at the application layer) qualify as two distinct resources. The twelve-factor app treats these databases as *attached resources*, which indicates their loose coupling to the deploy they are attached to.



Resources can be attached to and detached from deploys at will. For example, if the app's database is misbehaving due to a hardware issue, the app's administrator might spin up a new database server restored from a recent backup. The current production database could be detached, and the new database attached – all without any code changes.

- My code has shown encapsulating backing services with a shallow adaptor.
- This is an example of the *adaptor pattern*.



- You seen examples of things like `AbstractBaseDataService`,
- The approach is also an application of several aspects of SOLID.
(<https://en.wikipedia.org/wiki/SOLID>)

Solid Principles

SOLID Principles

blog.bytebytego.com

S

Single Responsibility Principle (SRP)

A class should have only one reason to change, meaning it should have a single, well-defined responsibility.

O

Open/Closed Principle (OCP)

Software entities (e.g., classes, modules) should be open for extension but closed for modification. This promotes the idea of extending functionality without altering existing code.

L

Liskov Substitution Principle (LSP)

Subtypes (derived classes) must be substitutable for their base types (parent classes) without altering the correctness of the program.

I

Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they don't use. This principle encourages the creation of smaller, focused interfaces.

D

Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. This promotes the decoupling of components through abstractions and interfaces.

Microservices Guidelines

SOLID Principles

Loosely coupled	Interface Segregation + Dependency Inversion
Testable	Single Responsibility + Dependency Inversion.
Composable	Single Responsibility + Open/close
Highly maintainable	Single Responsibility + Liskov Substitution + Interface Segregation
Independently deployable	Single Responsibility + Interface Segregation + Dependency Inversion
Capable of being developed by a small team	Single Responsibility + Interface Segregation

<https://medium.com/@saurabh.engg.it/microservices-designing-effective-microservices-by-following-solid-design-principles-a995c3a033a0>

This is a good overview article, and we will see patterns in later lectures.

- This slide and the preceding slides could have been a 60 slide presentation.
- If you put 3 SW architects into a room, you will get 15 design principles and patterns.
- Use common sense.

Hexagonal Architecture

Hexagonal Architecture

- The hexagonal architecture, or ports and adapters architecture, is an architectural pattern used in software design. It aims at creating loosely coupled application components that can be easily connected to their software environment by means of ports and adapters. This makes components exchangeable at any level and facilitates test automation
- The hexagonal architecture divides a system into several loosely-coupled interchangeable components, such as the application core, the database, the user interface, test scripts and interfaces with other systems. This approach is an alternative to the traditional layered architecture.
- Each component is connected to the others through a number of exposed "ports". Communication through these ports follow a given protocol depending on their purpose. Ports and protocols define an abstract API that can be implemented by any suitable technical means (e.g. method invocation in an object-oriented language, remote procedure calls, or web services).

